

Bloque B — Especificación Técnica

Inteligencia Artificial y Normativa de UYW4Q

1. Alcance del bloque

Este bloque recibe el JSON de hallazgos técnicos que produce el Bloque A ((dns_findings), (ssl_findings), (breach_findings)) y lo transforma en algo que un dueño de empresa sin conocimiento técnico pueda entender: **qué tan grave es, por qué le importa a SU negocio específico, y qué tiene que hacer al respecto según la ley peruana.**

No construye ningún hallazgo nuevo — solo interpreta, prioriza y contextualiza lo que el Bloque A ya detectó de forma determinística. Esto es clave recordarlo: si el jurado o el profesor preguntan "¿qué pasa si la IA se equivoca?", la respuesta es que la capa de IA nunca inventa un hallazgo técnico, solo lo interpreta — la base siempre es verificable.

2. Componentes

2.1. Base de conocimiento normativo (se construye ANTES de tocar el LLM)

Esto es investigación y curación manual, no código — y es la parte más lenta del bloque, no la subestimen:

- Mapeo **artículo por artículo** del D.S. 016-2024-JUS: qué exige cada artículo relevante (documento de seguridad, consentimiento explícito, evaluación de impacto, designación de ODP, notificación de 48 horas) y qué hallazgo técnico se relaciona con cada uno.
- Tabla de **casos peruanos reales documentados** (Interbank, Topitop, UPC) con una descripción breve de cada uno, para que el motor pueda citarlos como referencia cuando sea relevante — con cuidado de no reproducir contenido extenso de las fuentes, solo el hecho y la cifra.
- Rango de multas por categoría de infracción (leve, grave, muy grave) con los montos exactos en soles y UIT.

Formato de almacenamiento sugerido: un archivo estructurado (JSON o YAML) que el LLM consulta como contexto en cada llamada — no confíen en que el modelo "ya sabe" la ley peruana de memoria; désela explícitamente en cada consulta (esto se llama *retrieval-augmented generation* o RAG a nivel básico, no necesitan una base vectorial compleja para el volumen de información que manejan, un archivo estructurado bien armado alcanza para el MVP).

2.2. Prompt engineering — motor de priorización de riesgo

El prompt recibe:

- El JSON de hallazgos técnicos del Bloque A.
- El rubro declarado por el cliente (clínica, e-commerce, academia, agencia, delivery).
- El fragmento relevante de la base de conocimiento normativo (solo los artículos que aplican a los hallazgos detectados, no toda la ley completa en cada llamada — esto reduce costo y mejora precisión).

Y debe devolver **un JSON estructurado**, nunca texto libre sin formato — esto es importante para que el Bloque C (frontend) pueda renderizarlo de forma consistente:

```
json
{
  "risk_score": 68,
  "risk_level": "alto",
  "priority_findings": [
    {
      "finding": "DMARC ausente",
      "business_risk": "Tu dominio puede ser usado para suplantar tu marca en correos de phishing dirigidos a tus pacientes.",
      "legal_reference": "Art. X del D.S. 016-2024-JUS - medidas técnicas de seguridad",
      "urgency": "alta"
    }
  ],
  "summary_for_owner": "Texto en español simple, 2-3 líneas, dirigido al dueño del negocio, no a un técnico."
```

2.3. Generador de plantilla de notificación de brecha

Se activa solo cuando el cliente reporta o confirma un incidente real (no en cada escaneo rutinario). Toma los datos ya disponibles del sistema (dominio, hallazgos previos, tipo de datos que maneja según el rubro) y arma un borrador que responde a los campos exactos del **Formulario Virtual Único de Registro de Incidentes de Seguridad de la Información** del CNSD (ya verificado en la sección legal): sistemas afectados, tipo de datos comprometidos, número estimado de personas afectadas, cronología.

Importante (ya lo definieron en la conversación con el profesor): este generador prepara el contenido, no lo envía — el cliente copia y confirma manualmente en el portal oficial (reporte.cnsd.gob.pe/home/minjus). El sistema puede, como automatización interna válida, enviar un correo al cliente con este borrador y el link directo al formulario — eso sí es automatización aceptable porque no toca el sistema del Estado, solo el flujo interno de UYW4Q.

3. Selección del proveedor de LLM

Punto pendiente de decisión — criterios a evaluar antes de comprometerse:

Criterio	Por qué importa
Calidad de razonamiento en español	El público objetivo (dueños de MYPEs/medianas) necesita explicaciones claras, no traducciones torpes
Costo por consulta	Con escaneos frecuentes por cliente en el plan de suscripción, el costo se acumula — hagan una proyección de costo mensual estimado según el número de clientes proyectado
Capacidad de seguir un formato JSON estructurado de forma confiable	Necesitan que la salida sea parseable por el backend sin fallos frecuentes
Latencia	Si la demo en vivo depende de una respuesta rápida, un modelo lento puede arruinar el momento del pitch

Recomendación práctica: hagan pruebas comparativas con 2-3 proveedores usando el mismo prompt y los mismos hallazgos de ejemplo, antes de comprometerse con uno solo — no decidan por preferencia personal sin comparar resultados reales.

4. Mitigación de riesgo de alucinación (van a necesitar esto para la reunión con el profesor y para el jurado)

1. **Separación estricta entre hallazgo (determinístico) e interpretación (IA):** el LLM nunca decide si SPF está mal configurado — eso ya lo determinó el Bloque A con una consulta DNS verificable. El LLM solo interpreta lo que ya es un hecho.
2. **Contexto normativo inyectado explícitamente**, nunca dependiendo de que el modelo "recuerde" la ley peruana de memoria — esto reduce drásticamente el riesgo de que invente un artículo o una cifra de multa incorrecta.
3. **Validación de formato de salida:** si el LLM no devuelve el JSON en el formato esperado, el sistema debe manejarlo con un reintento o un mensaje de error controlado — nunca mostrarle al cliente una salida rota o a medio generar.
4. **Descargo de responsabilidad explícito en el producto:** "diagnóstico orientativo, no dictamen legal vinculante" — esto no es solo protección legal para UYW4Q, es honestidad real sobre los límites de la herramienta.
5. **Pruebas manuales con casos conocidos antes del lanzamiento:** correr el motor contra 10-15 combinaciones de hallazgos ya conocidos y verificar manualmente que la salida sea razonable, antes de confiar en él para la demo.

5. Estructura de carpetas (se integra dentro del proyecto backend del Bloque A)

```
app/
├── core/
│   ├── risk_scoring.py      # ya existe en Bloque A, aquí se conecta con...
│   └── ai/
│       ├── llm_client.py    # cliente del proveedor de LLM elegido
│       ├── prompt_builder.py # arma el prompt con hallazgos + rubro + normativa
│       ├── response_parser.py # valida y parsea el JSON de salida
│       └── incident_template.py # generador de la plantilla de notificación
├── knowledge/
│   ├── normativa_ds016.json # base de conocimiento curada del reglamento
│   └── casos_peru.json      # casos documentados (Interbank, Topitop, UPC)
└── tests/
    └── test_ai_risk_scoring.py # pruebas contra casos conocidos (sección 4, punto 5)
```

6. Orden de construcción recomendado

1. **Semana 1-2 (en paralelo al Bloque A):** investigación y curación de la base de conocimiento normativo — esto no depende de tener el backend listo, puede arrancar de inmediato.
2. **Semana 2:** definir y probar el prompt con datos de ejemplo (mockeados, sin esperar al Bloque A real).
3. **Semana 3:** integración real con la salida del Bloque A en cuanto esté disponible.
4. **Semana 3-4:** generador de plantilla de notificación de brecha.
5. **Semana 4:** batería de pruebas contra casos conocidos (sección 4, punto 5) — no dejen esto para el último día antes de Finales Locales.

7. Riesgos a vigilar

- **Costo acumulado de llamadas al LLM** si no se cachean resultados — un mismo dominio escaneado varias veces no debería regenerar la interpretación de IA desde cero cada vez si los hallazgos no cambiaron.
- **Dependencia de disponibilidad del proveedor de LLM el día de la demo** — tengan un plan B (capturas de pantalla de una corrida exitosa previa) por si el servicio falla justo durante Finales Locales.
- **Deriva de la base normativa:** si la ley o el reglamento cambian, la base de conocimiento queda desactualizada — no es un problema para el MVP del concurso, pero vale la pena mencionarlo como parte del roadmap de mantenimiento si preguntan por sostenibilidad a largo plazo.